Solver-Verified Text-Conditioned Sokoban Level Generation: Autoregressive Models Can Count, One-Shot Decoders Cannot

Aayush Tiwari
Independent
aayushkumar345@gmail.com

***Abstract*—We compare three generative model families on text-conditioned Sokoban level generation and evaluate them with an exhaustive A* solver rather than a learned or heuristic proxy. Because the state space of a 10×10 Sokoban level is bounded and every prune we use is sound, exhausting the search is a *proof* of unsolvability: the verifier is a decision procedure, not a one-sided test. We therefore report three outcomes — solved, proven unsolvable, and timed out — rather than two. Our central finding is that autoregressive generation satisfies hard counting constraints that one-shot generators cannot: a 10.7M-parameter decoder-only transformer emitting a level cell by cell reaches 82.6% structural validity unaided, while a conditional VAE decoding all 100 cells from a latent vector produces zero valid levels in $100{,}000$ draws under argmax decoding, because the argmax of a product of independent per-cell marginals annihilates exactly the rare tiles the constraints are about. Constrained decoding closes the counting gap deterministically, at 100% validity. Solvability, however, does not follow from validity: an empty-room baseline with no interior walls is solvable 23.4% of the time against the transformer's 49.6%, so the gap is real but solvability alone is not a sufficient objective — a retrieval baseline that copies training levels maximises it at zero novelty. Two results cut against the simple story: repairing the VAE's counting errors yields 76.6% solvability, *above* the transformer, locating the autoregressive advantage in counting rather than in spatial structure; and every family fails to extrapolate, producing ~ 30-move levels when asked for ~ 116. We argue the results table must be read as a joint constraint over solvability, difficulty control and novelty. This work replicates and extends Todd et al. [1]; the data-scaling and tokenization findings are theirs, and we claim neither.**

***Index Terms*—procedural content generation, generative models, constrained decoding, verification, Sokoban**

I. Introduction

Evaluating a generative model is usually a matter of proxies. Perplexity does not tell you whether a program runs; a discriminator score does not tell you whether a level can be finished. The situation is better than it is sometimes described — HumanEval executes unit tests [2], Lean checks proofs [3], and text-to-SQL is scored by execution accuracy [4] — so it is simply false that no metric can tell you whether a generated artifact works.

What those verifiers share is that they are *sound but incomplete*. A unit test that passes does not prove a program correct, and a test suite cannot prove a program wrong: it can only fail to find a witness. The verifier answers "yes" reliably and "no" only provisionally.

Sokoban is unusual in admitting the other kind of verifier. The state space of a 10×10 level with four boxes is finite and small enough to exhaust. Given pruning rules that are *sound* — they discard only states from which no solution exists — running the open set to empty is a constructive proof that no solution exists. The verifier is a **decision procedure**. This is why we report a three-way outcome; collapsing "timed out" into "unsolvable" would convert an absence of evidence into a false proof, in a direction that flatters the numbers.

The second question we ask is about model architecture. A valid level needs *exactly* one player, four boxes and four goals. A transformer that emits one cell at a time conditions each decision on how many boxes it has already placed; a VAE or GAN decoder that emits all 100 cells from a single latent vector cannot, because its per-cell outputs are conditionally independent given the latent. This is a structural prediction about which family can satisfy a counting constraint, and an exhaustive solver lets us measure both whether the constraint is met and whether the resulting level is any good.

a) Contributions:

1) An exhaustive, validated A* Sokoban solver used as a decision procedure, with a soundness ablation and a node-cap convergence curve (Section V).
2) A three-family comparison — from-scratch transformer, conditional VAE, conditional GAN — at roughly comparable parameter counts, with constrained decoding and repair reported as separate rows so that *counting* and *spatial understanding* are measured separately (Sections IV, VII).
3) Baselines that constrain the interpretation of the headline number: an open-room baseline that shows solvability is easy to obtain, and a retrieval baseline that shows it is trivially maximised by copying.
4) Data-quality findings about the standard Boxoban A* solution set that affect anyone using it for supervision (Section III).

We are explicit about provenance throughout. Claims are marked as **Tier 1** (proven by our experiments), **Tier 2** (justified by citation) or **Tier 3** (admitted defaults, such as untuned hyperparameters).

II. Related Work

a) LLMs for level generation: Todd et al. [1] fine-tune GPT-2 variants to generate Sokoban levels from a character encoding and study how solvability scales with training data and with tokenization. This paper is a replication and extension of that work; **the data-scaling result, the solvability-versus-conditioning setup, and the tokenization finding are theirs**, and we claim none of them. MarioGPT [5] conditions level generation on natural language for Super Mario Bros., where playability is checked by an A^* agent that is a sound but incomplete verifier — it can confirm a level is playable but cannot prove one is not.

b) Our delta: Relative to [1] we add three things: constrained decoding with feasibility forcing that makes structural validity deterministic rather than learned; three-way solver reporting that distinguishes proven unsolvability from timeout; and a controlled multi-family comparison at comparable parameter counts, including one-shot decoders and non-learned baselines.

c) PCGML: Procedural content generation via machine learning [6] has long used autoencoders and GANs over tile grids. Our contribution is not a new architecture but a verification methodology: the same tile-grid models, judged by a decision procedure instead of a proxy.

d) Sokoban and search: Sokoban is PSPACE-complete in general [7], and the solver literature is built on domain-specific pruning [8]. Our solver is deliberately conservative: only two prunes, both sound, because an unsound prune produces false proofs of unsolvability. The Boxoban corpus [9] was introduced for model-free planning research and generated by the procedure of Racanière et al. [10]; its test split is the one used by Orseau et al. [11].

III. Dataset

We use the Boxoban `unfiltered` split [9]: 100,000 training levels, 5,000 from the *official* validation split (we do not carve a validation set out of train), and the 1,000-level test split held frozen for solver validation and as the real-level ceiling row. Solution lengths come from the A* solution set of Garriga-Alonso et al. [12].

A. Verified format properties

We verified rather than assumed every property the representation depends on (Tier 1). Over 50 level files the tile alphabet is exactly $\{\#, @, \$, ., \text{␣}\}$; in particular **neither ⋆ (box on goal) nor + (player on goal) occurs anywhere**, so a box never *starts* on a goal. That single fact is what makes the five-channel one-hot tensor, the six-symbol grid vocabulary, and the "exactly four `$` and four `.`" constraint all well-defined; had it failed, each would have broken silently. Over 10,000 levels, every grid is 10×10 with a complete wall border and exactly one player, four boxes and four goals, with zero violations. Files use CRLF line endings, which the specification did not anticipate and the parser normalises. There are zero exact collisions between the training levels and either the test or validation split.

B. Solution lengths count moves, not pushes

The solution set stores an action string over $\{0, 1, 2, 3\}$ = Up, Right, Down, Left, and a `Steps` field equal to its length. `Steps` therefore counts **player moves, not pushes** (Tier 1). We confirmed the action encoding by replaying: under this convention the actions reach the goal state exactly, and no other permutation of the four directions, with or without transposition, replays even one of the levels that fail. Their search minimised moves, so `Steps` is move-optimal where it is valid. This distinction sets the units of every caption and of the controllability metric.

C. A data-quality finding

The solution set documents rows whose action field is `SEARCH_STATE_FAILED` or `NOT_FOUND`. We additionally found rows that *look* well-formed — a plain digit string — but **do not replay to a solved state**: $63/1000$ (6.3%) of `unfiltered_test` and approximately 0.7% of `unfiltered_train`. They fail by walking into a wall or making an impossible push part-way through, and treating blocked moves as no-ops does not rescue any of them. We therefore **replay-validate every solution** and drop levels whose solution does not replay (0.73% of train, 7.4% of validation and test). Anyone using this set for supervision should do the same; a caption built on a wrong length is worse than no example.

D. Captions

Five features are computed from the grid; nothing is hand-labelled. Difficulty is the solution length binned by the terciles of the training distribution ($\leq$ 26, $\leq$ 35, $>$ 35 moves); solution length is quoted to the nearest five moves; wall density is the interior wall fraction split at the training median (0.500); connectivity is the mean number of non-wall neighbours per floor cell split at the training median (2.889); and box clustering is the mean pairwise Manhattan distance between boxes split at the training median (3.500). A caption reads, for example, `"hard, ~40 moves, dense walls, corridors, boxes scattered"`.

The connectivity feature was chosen empirically. The natural first choice, counting connected floor components, is **constant** on this corpus: measured over 20,000 training levels every Boxoban level has exactly one connected floor region, so the feature carries no information and cannot bin anything. We use mean floor degree instead, which ranges 2.19 to 3.36.

The same five features are encoded twice: as caption text for the transformer, and as a 10-dimensional vector for the VAE and GAN. **These are different information channels**, so the family comparison is not perfectly controlled on conditioning; we state this rather than claim otherwise.

IV. Method

A. Transformer (primary)

A decoder-only transformer written from scratch: $d_{\text{model}} = 384$, 6 layers, 6 heads, $d_{\text{ff}} = 1536$, learned positional embeddings, pre-norm, GELU, dropout 0.1, tied input/output embeddings — 10,734,336 parameters. We deliberately do *not* use a fine-tuned pretrained model as the primary system, so that the family comparison is roughly parameter-fair (10.7M against 1.1M and 1.1M) rather than 82M against 1M. The vocabulary is 48 hand-written characters; the sequence is <bos> caption <sep> grid(110 chars) <eos>, and the loss is masked over the caption prefix so the model is trained to predict the grid, not the caption.

B. Constrained decoding

A manual sampling loop applies a per-step logit mask. Border cells are forced to wall; the eleventh character of each row is forced to newline; and any tile whose quota is exhausted is masked. The rule that matters is **feasibility forcing**: letting $r = (1 - n_{\text{player}}) + (4 - n_{\text{box}}) + (4 - n_{\text{goal}})$ be the specials still owed and ℓ the interior cells remaining, when $\ell \leq r$ we mask # and space so that only specials can be emitted. Masking upper bounds alone enforces "at most four boxes", not "exactly four". With nine specials in 64 interior cells this rule rarely binds, which is precisely the trap: an implementation without it looks correct on a handful of samples and fails in the tail.

We instrument the rule and report the **forced-token rate**, which is a free measure of how well the model learned to count unaided.

C. Conditional VAE

An MLP encoder and decoder over a $5 \times 10 \times 10$ one-hot tensor, latent dimension 32, 1,098,292 parameters. We use MLPs rather than convolutions because 10×10 does not factor cleanly for strided convolutions and at 100 cells convolutions buy nothing while costing shape bugs. The reconstruction loss is **per-cell categorical cross-entropy over the channel dimension**, not MSE: MSE on one-hot tiles treats a categorical axis as a metric space, and is a common and wrong default. Posterior collapse is prevented with free bits [13], clamping per-dimension KL at a 0.05-nat floor.

We report **two decoding protocols, both mandatory**. The decoder emits an independent categorical per cell, and boxes occupy $4/100$ cells, goals $4/100$ and the player $1/100$. Under argmax, floor or wall therefore wins almost everywhere and the output is a near-empty room: the argmax of a product of marginals is not the mode of the joint, and rare tiles are exactly what gets annihilated. That is a correct and interesting result, but reporting argmax alone would be an unfair protocol, so we also report per-cell sampling, which preserves marginal tile rates and produces roughly the right *number* of boxes in the wrong *places* — a different failure signature.

D. Conditional GAN

An MLP generator and discriminator, 1,082,357 parameters combined, with straight-through Gumbel-softmax [14] discretisation annealed from temperature 1.0 to 0.3, and one-sided label smoothing. Training on continuous logits and taking argmax only at sampling time is not a valid option: the discriminator can then separate real one-hot tensors from soft generator output purely by per-cell entropy, training collapses to that trivial discriminator, and the generator learns nothing about layout. This family was hard time-boxed; mode collapse is reported as a finding, not treated as a failure to fix.

E. Repair

Any structurally invalid grid is projected onto the nearest valid grid using the model's own predicted probabilities: the border is forced to wall, excess boxes, goals and players are dropped lowest-probability first, deficits are filled highest-probability first, and specials are never placed on the border. Repair is idempotent and is the identity on already-valid grids. Reporting post-repair solvability decomposes the comparison into two separable questions: *can the model count?* (structural validity) and *does it understand spatial structure?* (post-repair solvability). Without it a reader cannot tell whether the GAN is bad at Sokoban or merely bad at arithmetic.

F. Baselines

Random placement (uniform interior walls at the training-median density, then random specials) is the floor. **Open room** — border walls only, zero interior walls — is the baseline that can invalidate the headline number, since an empty room has no interior walls to freeze a box against. Rule-based generation adds a floor-connectivity check and places boxes and goals only on live squares. **Retrieval** samples a training level verbatim, and exists to make the complementary point: solvability alone is a degenerate objective, maximised by copying. Real held-out Boxoban levels give the ceiling row.

G. Comparison protocol

The transformer is reported **both with and without** constrained decoding, so a reader can compare unconstrained-to-unconstrained across all three families and see the constrained result separately. Structural validity and solvability are never merged into one number: merging them compares decoding schemes rather than model families.

V. Solver

A. Design

Levels are represented as integer bitmasks (walls, goals, boxes) plus a player cell index. Successors are **pushes**, not single steps. For hashing, the player's exact cell is irrelevant — only its reachable region matters — so the state key is (minimum reachable cell, box mask). Without this canonicalisation, states differing only by player position within one region are distinct and the search inflates by roughly the region size; measured on the 1,000 real test levels, the starting region averages 19.1 cells (median 22.5, maximum 46).

The heuristic precomputes, per goal, a backward BFS map of the minimum pushes to bring a box to that goal, ignoring other boxes but respecting walls, and takes the minimum-cost assignment of boxes to goals [15]. It is admissible — each push moves one box one cell — and strictly tighter than sum-of-Manhattan because it respects walls.

B. Two sound prunes, and nothing else

A cell is *dead* if a box there can reach no goal under the relaxation above; any box on a dead square is a proven deadlock, which subsumes corner deadlocks. A box is *frozen* if it is blocked along both axes, where a blocker is a wall, a dead square, or another frozen box, with a currently-evaluated box treated as blocking to terminate the recursion; any frozen box off a goal is a proven deadlock. We admit nothing heuristic: **an unsound prune produces false proofs of unsolvability**, which is the worst failure mode available here because it corrupts the headline number invisibly and in a plausible direction.

`unsolvable` is returned **only** when the open set empties before any budget is hit. Every cap hit is `timeout`.

C. Validation

On all 1,000 real test levels the solver achieves a **100.00% solve rate** with zero `unsolvable` verdicts and zero timeouts (Tier 1). Every one of the 1,000 returned solutions replays to a solved state under an independent simulator. Against the published solutions, our push-optimal length is $\leq$ theirs on $926/926$ comparable levels and our reconstructed move length is $\geq$ their move-optimal length on $926/926$, both as theory requires.

The **soundness ablation** reruns a 200-level sample with both prunes disabled and a $10\times$ node cap: **zero verdict disagreements and zero length disagreements**. We extended this to 450 levels drawn from the three baseline generators, whose wall densities and failure modes differ sharply from real levels, again with zero disagreements. Median expansions are 176 (p99 3,458) and median wall time 75 ms (p99 2.0 s).

Solve rate reaches 100% at a node cap of 10,000 and is flat to 1,000,000, so the reported numbers are converged rather than budget-limited.

D. Move length is an upper bound

Because push-optimal search discards the player's exact position, the move length reconstructed by walking the player between consecutive pushes is an **upper bound**, not the optimum. Measured against exact move-optimal search on 196 levels: mean excess $+40.5\%$, exactly optimal on only 6.6% of levels, and zero bound violations. Because that gap is large, the achieved solution length used in the controllability metric is measured with the **exact move-optimal solver**, matching the move units of the captions, rather than with the cheap reconstruction.

VI. Experiments

For each condition we draw samples until 500 structurally valid levels are collected or a hard cap of 100,000 draws is reached, and we report **samples drawn** as its own column. Only valid samples are handed to the solver, so drawing tens of thousands of one-shot samples costs seconds and no solver time. Where a family cannot reach 500 valid levels within the cap we report the count obtained and state the shortfall; we never report a "solvable given valid" percentage on a denominator below 30 without flagging it inline, because a family at 2% validity yields roughly ten levels and a percentage there is noise a reader will misread as signal.

All proportions carry Wilson score intervals [16], which behave correctly near 0% and 100%. Pairwise comparisons use two-proportion z-tests. Three seeds are run on the main transformer configuration; every other row is single-seed and declared as such in the table caption (Tier 3).

Controllability uses a fixed prompt suite stored in the repository: five solution-length targets $\times$ two density settings $\times$ two connectivity settings $\times$ two clustering settings $\times$ 15 samples $= 600$ prompts. Wall density is a two-bin feature by construction, so a balanced factorial is used in place of three density settings, keeping the 600-prompt total while letting every conditioned attribute vary independently. A separate out-of-distribution suite requests 90, 110 and 150 moves; on the training split the 99th percentile is 67 moves, the 99.9th is 87, and the maximum is 130.

VII. Results

The main results appear in Table I. We read them as five findings.

A. Autoregressive models can count; one-shot decoders cannot

The counting gap is the largest effect in the table, and it is structural rather than a matter of capacity. The transformer reaches 82.6% [80.2, 84.8] structural validity with no constraint at all, needing 1,024 draws to yield 500 valid levels. The conditional VAE under argmax decoding yields **zero** valid levels in 100,000 draws. Sampling the VAE per cell instead of taking the argmax raises validity only to 1.7% [1.6, 1.9], requiring 29,184 draws; the GAN reaches 8.6%.

Figure 1 shows why, more convincingly than any percentage. The transformer's box-count distribution is a spike at four (87.1% exactly four, mean 3.87 ± 0.34). The VAE under argmax has mean 1.38 ± 1.23 boxes and 0.01 ± 0.12 goals: the per-cell argmax of a product of marginals picks floor or wall almost everywhere, because boxes occupy $4/100$ cells and goals $4/100$. Per-cell *sampling* restores the marginal rates almost exactly — mean 3.95 ± 1.72 boxes, 4.05 ± 1.85 goals — but only 21.8% of samples carry exactly four boxes. That is the signature of the failure: the one-shot decoder gets the right *average* number of boxes and the wrong *joint*, because its per-cell outputs are conditionally independent given the latent.

Constrained decoding closes the gap deterministically at 100% validity, and with a very light touch: feasibility forcing

TABLE I

STRUCTURAL VALIDITY AND SOLVER-VERIFIED SOLVABILITY ACROSS MODEL FAMILIES. PERCENTAGES CARRY WILSON 95% CONFIDENCE INTERVALS, WHICH CAPTURE SAMPLING VARIATION ONLY. STRUCTURAL VALIDITY AND SOLVABILITY ARE REPORTED SEPARATELY AND NEVER MERGED. **EVERY ROW IS A SINGLE TRAINING RUN.** THE PRIMARY TRANSFORMER CONFIGURATION WAS ADDITIONALLY RETRAINED WITH THREE SEEDS, GIVING A ± 7.0 POINT SPREAD ON SOLVABILITY; DIFFERENCES BELOW ROUGHLY 15 POINTS BETWEEN SINGLE-SEED ROWS ARE THEREFORE NOT ESTABLISHED.

Model	Valid %	Solv. %	Solv. | valid %	Timeout %	Draws	Novel %	Diversity	Params
Random placement	100.0 [99.3, 100.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.8]	0.0	512	100.0	39.20	–
Open room	100.0 [99.3, 100.0]	23.4 [19.7, 27.1]	23.4 [19.9, 27.3]	0.0	512	100.0	16.22	–
Rule-based	100.0 [99.3, 100.0]	0.4 [0.0, 1.0]	0.4 [0.1, 1.4]	0.0	512	100.0	38.14	–
Retrieval	100.0 [99.3, 100.0]	100.0 [100.0, 100.0]	100.0 [99.2, 100.0]	0.0	512	0.0	38.73	–
Conditional GAN (raw)	8.6 [8.0, 9.4]	4.3 [3.8, 4.9]	50.2 [45.8, 54.6]	0.0	6,144	100.0	34.18	1,082,357
Conditional GAN (repaired)	100.0 [99.3, 100.0]	40.2 [35.9, 44.5]	40.2 [36.0, 44.6]	0.0	512	100.0	36.79	1,082,357
Conditional VAE (argmax)	0.0 [0.0, 0.0]	–	–	–	100,000	–	–	1,098,292
Conditional VAE (sampled)	1.7 [1.6, 1.9]	0.5 [0.4, 0.6]	30.0 [26.1, 34.2]	0.0	29,184	100.0	38.54	1,098,292
Conditional VAE (repaired)	100.0 [99.3, 100.0]	76.6 [72.9, 80.3]	76.6 [72.7, 80.1]	0.0	512	100.0	37.82	1,098,292
Transformer (unconstrained)	82.6 [80.2, 84.8]	46.3 [42.4, 50.1]	56.0 [51.6, 60.3]	0.0	1,024	100.0	39.01	10,734,336
Transformer (constrained)	100.0 [99.3, 100.0]	49.6 [45.2, 54.0]	49.6 [45.2, 54.0]	0.0	512	100.0	38.86	10,734,336
DistilGPT-2 (constrained, ablation)	100.0 [99.3, 100.0]	63.6 [59.4, 67.8]	63.6 [59.3, 67.7]	0.0	512	100.0	38.69	43,352,064
Real Boxoban levels	100.0 [99.3, 100.0]	100.0 [100.0, 100.0]	100.0 [99.2, 100.0]	0.0	512	100.0	38.76	–

fired on 25.0% of levels and supplied a mean of only 0.378 forced tokens per level. The model had largely learned to count unaided; forcing supplies the last box.

B. Solvability does not follow from validity

Every non-learned baseline is 100% structurally valid by construction and still nearly worthless. Random placement is solvable 0.0% of the time and the rule-based generator 0.4%, despite the latter enforcing floor connectivity and placing boxes only on live squares. Interior walls are what kill them: at the training-median density the floor becomes a maze of narrow corridors, and a box pushed into one freezes against the walls.

The **open-room** baseline is the row that constrains how the headline number may be read. With no interior walls at all it is solvable 23.4% [19.9, 27.3] of the time. The transformer's 49.6% is significantly higher ($+26.2$ percentage points, $z = 8.60$, $p < 10^{-16}$), so the headline is not an artifact of the task being easy — but a reader given only "49.6% solvable" without the open-room row would substantially overestimate what has been shown.

The **retrieval** baseline makes the complementary point: 100% solvable, 0% novel, nearest-neighbour distance exactly 0.00 to the training set. Solvability alone is a degenerate objective, maximised by copying. The table has to be read as a joint constraint over solvability, difficulty control and novelty; no single column of it is an objective.

C. Repair separates counting from spatial understanding

The most surprising row is the repaired VAE. Projecting its invalid output onto the nearest valid grid yields 76.6% [72.7, 80.1] solvability, well *above* the transformer's 49.6%, with diversity (37.8) close to the real-level reference (38.8) and no exact copies. The repaired GAN reaches 40.2%.

This is exactly what the repair experiment was for. The VAE is not bad at Sokoban; it is bad at arithmetic. Its per-cell marginals encode where boxes and goals plausibly go, and repair reads those marginals off and places the right number of tiles at the most probable positions. What autoregressive generation buys is the ability to satisfy a counting constraint *while sampling*, not a better model of spatial structure. Merging structural validity and solvability into one number would have hidden this result completely.

A second, smaller inversion points the same way: the *unconstrained* transformer is more solvable given validity (56.0%) than the constrained one (49.6%; $z = -2.03$, $p = 0.043$). Unconstrained sampling keeps only the 82.6% of levels the model got right unaided, which are its more typical and more solvable outputs; constrained decoding additionally rescues the levels it would have botched, and those are worse levels. This is the distribution shift predicted in Section IV-B appearing as a measurable cost, and it is why both rows are reported. On the overall rate — solvable out of all samples drawn — constrained decoding still wins, 49.6% against 46.3%.

D. Controllability splits along a predictable line

Control is reported per attribute, never as one number, and the split is sharp. The transformer hits the caption's wall-density bin 88.8% of the time, connectivity 80.0% and box clustering 76.3%, against a 50% chance rate confirmed by two unconditioned controls (real levels score 48.8%, 53.2% and 52.2%). But difficulty — the binned solution length — is hit only 43.3% of the time against a 33% chance rate, and the rank correlation between requested and achieved solution length is $\rho = 0.484$.

The dividing line is the one we predicted before running the experiment. Wall density, connectivity and clustering are computable directly from the grid, so a model that reproduces surface statistics gets them nearly for free. Solution length is not computable without search. It is the only conditioned attribute that requires the model to have learned something about the puzzle rather than about its pixels, and it is the one the model controls worst.

That correlation is also **censored**: 55.2% of suite samples have no achieved length, because they were proven unsolvable, and the surviving subsample is biased toward easy levels —

the direction that inflates ρ. The honest reading is that 0.484 is an upper bound on a weak effect.

E. Seed variance is large, and it bounds what can be claimed

We trained the primary transformer configuration three times, changing only the seed, and sampled and solved each identically. Structural validity is perfectly stable at $100.0\% \pm 0.0$ — unsurprisingly, since constrained decoding guarantees it — and diversity is stable at 38.60 ± 0.31. But solvability-given-validity is **not**: 46.6%, 34.6% and 47.0%, giving $42.7\% \pm 7.0$. Validation loss barely moves across the same three runs (0.357 ± 0.005), so validation loss is a poor predictor of the quantity we actually care about.

This number governs how firmly the rest of the table can be read, and we prefer to state it plainly rather than lean on a single favourable run:

- The comparison against the baselines survives easily. Even the worst of the three seeds (34.6%) is far above open room (23.4%) and rule-based (0.4%).
- The comparison against DistilGPT-2 does **not** survive. Its 63.6% sits roughly three standard deviations above the from-scratch mean, but it is a *single* run being compared against a distribution with ± 7 point spread, and we did not train it three times.
- The constrained-versus-unconstrained gap (6.4 points) is smaller than the seed spread. It is a within-checkpoint decoding comparison, so training-seed variance does not apply to it directly, but it is measured on one model and should be treated as indicative.

Every other row in this paper is single-seed and its caption says so. Given a ± 7 point spread on the headline metric, no row-to-row difference under about 15 points should be read as established.

F. Pretraining appears to transfer across a vocabulary boundary

The DistilGPT-2 ablation discards the 50257×768 embedding matrix and the tied LM head, so nothing lexical can transfer. It nonetheless reaches 63.6% [59.3, 67.7] solvability against the from-scratch model's 49.6%, needs less rescuing from the constrained decoder (7.4% of levels against 25.0%), and controls solution length better ($\rho = 0.673$ against 0.484). The suggestion is that pretrained sequence-modelling machinery transfers to a domain with zero shared vocabulary.

We report this as suggestive, not established. The ablation is single-seed and the from-scratch model's own seed spread is ± 7 points (Section VII-E), so a 14-point difference between one run of each is not conclusive. The cost side is unambiguous: $4.0\times$ the parameters (43.4M against 10.7M) and $6.8\times$ the training time (4479 s against 661 s).

G. Temperature trades solvability against diversity

Figure 2 shows a clean monotone trade-off: solvability falls from 61.2% at $T = 0.6$ to 34.8% at $T = 1.5$, while diversity rises from 35.8 to a peak of 39.0 at $T = 1.2$. $T = 1.5$ is Pareto-dominated — it loses solvability without buying diversity back. At $T = 1.0$ the model already matches the real-level diversity reference (38.9 against 38.8).

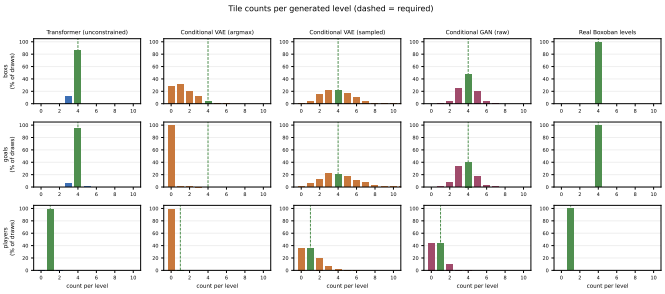

Fig. 1. Tile counts per generated level over raw draws, valid or not. The dashed line marks the required count. The transformer's distribution is a spike at four; the VAE under argmax collapses to a near-empty room; VAE sampling recovers the right marginal rate with the wrong joint.

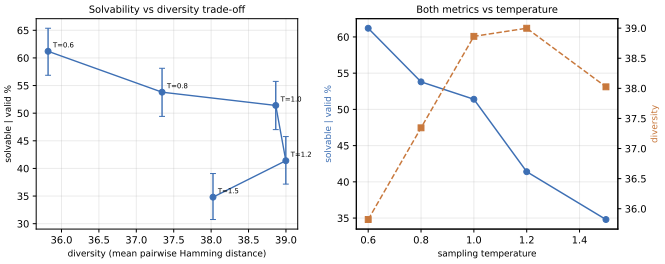

Fig. 2. Solvability against diversity for the constrained transformer across sampling temperatures. Bars are Wilson 95% intervals.

VIII. Failure Analysis

a) Extrapolation fails completely: The out-of-distribution suite requests solution lengths of 90, 110 and 150 moves, against a training distribution whose 99th percentile is 67 and whose maximum is 130. Every family ignores the request. The constrained transformer produces levels averaging 29.5 achieved moves when asked for 115.8, with a rank correlation of -0.394; the repaired VAE averages 42.3. Real Boxoban levels sampled against the same captions average 30.3, which is the correct null — they are not conditioned at all. Models reproduce the training length distribution regardless of what the caption asks. Length conditioning learned in-distribution does not extend past the data.

b) The one-shot failure mode is a plausible, empty room: The VAE's argmax output is not noise. It is a clean, empty, unplayable room with roughly one box and no goals whatsoever. It fails the constraint in the most systematic way available, and it fails *silently*: nothing about the output looks malformed, there is simply no puzzle in it. A soft metric comparing tile histograms in aggregate would not obviously flag this, which is a small argument for verification over proxies.

c) Hand-written domain knowledge is not a shortcut: The rule-based generator — connectivity checks plus live-square placement — achieves 0.4% solvability, *worse* than an empty room. We checked two alternative explanations before

accepting that number. It is not an unsound prune: re-solving 450 baseline levels with deadlock pruning disabled at a $10\times$ node cap gives identical verdicts. And it is not caused by starting in a deadlock: additionally rejecting frozen start states moves the rate by less than one point. The levels die after the first push, not before it. Real Boxoban levels reach 100% at the same wall density only because they are built by reverse play from an already-solved state, which is a fundamentally different construction from sampling walls and then dropping boxes onto them.

d) Constrained decoding has a measurable cost: As reported above, forcing validity onto samples the model would otherwise have botched lowers solvability-given-validity by 6.4 points. Constrained decoding is not free; it trades sample quality for a guarantee. Whether that trade is worth making depends on which column of the table the application cares about.

IX. Limitations

- **Constrained decoding shifts the sampling distribution.** Mask-then-renormalise at each step does not sample from the model's distribution conditioned on the valid set; it is a greedy local approximation to it.
- **Conditioning channels differ across families.** The transformer is conditioned on caption text and the one-shot models on a 10-dimensional vector, so the comparison is not perfectly controlled on conditioning.
- **Reconstructed move length is an upper bound**, quantified in Section V at $+40.5\%$ mean excess. We avoid it where it matters by using exact move-optimal search for controllability.
- **A single game at a single fixed size**: 10×10, four boxes, four goals. Nothing here shows the finding generalises to other tile games or other grid sizes.
- **Large seed variance, single-seed ablations.** The main transformer configuration was run with three seeds and its solvability spread is ±7.0 points (Section VII-E); every other row is a single run. Differences smaller than roughly 15 points between single-seed rows are not established.
- **Untuned hyperparameters.** Learning rate, batch size and epoch count were chosen once and never swept, for any model (Tier 3). We did not try $1\mathrm{e}{-4}$ before settling on $3\mathrm{e}{-4}$.
- **Inverse-binomial sampling.** Because we draw until 500 valid levels are collected, structural-validity intervals are close approximations rather than exact fixed-n binomial intervals.
- **Censored controllability.** Achieved solution length exists only for solved levels, so the length correlation is computed on a subsample biased toward easy levels. We report the censoring rate alongside it.

X. Ethics

The models generate puzzle layouts for a single game. They consume no personal data, produce no text about people, and inform no decisions about people. The Boxoban corpus is procedurally generated and contains no human-authored or personal content.

The one substantive risk is misreading the verification claim. "Solver-verified" is a strong guarantee *only* within stated bounds — a 10×10 grid, four boxes, a $200{,}000$-node cap and a 10-second wall-clock cap. Outside those bounds a timeout proves nothing, which is exactly why timeouts are reported as a separate column rather than folded into "unsolvable". Presenting a bounded decision procedure as an unbounded guarantee would be the failure mode to avoid, and it is one that generative-model evaluation is generally prone to.

XI. Conclusion

We evaluated three generative model families on text-conditioned Sokoban level generation using an exhaustive A* solver as a decision procedure rather than a proxy, reporting solved, proven-unsolvable and timed-out separately. The solver itself is validated first — 100.00% solve rate on $1{,}000$ real levels, zero false proofs of unsolvability, every returned solution replaying to a solved state, and a soundness ablation with zero verdict disagreements across four distinct level distributions.

On that footing, three claims hold. Autoregressive generation satisfies hard counting constraints that one-shot decoders cannot: 82.6% structural validity unaided against *zero* valid levels in $100{,}000$ draws from a VAE under argmax, with constrained decoding closing the gap deterministically for a mean of 0.378 forced tokens per level. Solvability does not follow from validity, and the open-room baseline at 23.4% shows how much of a headline solvability number can come from the task rather than from the model. And the two axes come apart in the other direction as well: once its counting errors are repaired, the VAE is *more* solvable than the transformer, which locates the autoregressive advantage in counting rather than in spatial understanding.

We also measured how far these numbers can be trusted. Retraining the primary configuration with three seeds moves solvability by ±7.0 points while validation loss moves by ±0.005, so validation loss is a poor proxy for the property being evaluated, and single-seed comparisons between nearby rows — including our own DistilGPT-2 ablation — should not be read as settled.

The most interesting negative result is the controllability split. Attributes computable from the grid are controlled well; solution length, which requires search to evaluate, is controlled weakly ($\rho = 0.484$, on a censored subsample) and not at all out of distribution. A model conditioned on "hard" learns what hard levels *look like*, not what makes them hard — and that gap is invisible to any metric that does not run the game.

Reproducibility

All code, configurations, checkpoints, generated levels with per-level solver verdicts, and result JSON artifacts are released. Every artifact embeds its configuration, random seed and git commit; every table in this paper is regenerated from those artifacts by a single command, and no number was typed by hand.

REFERENCES

[1] G. Todd, S. Earle, M. U. Nasir, M. C. Green, and J. Togelius, "Level generation through large language models," in *Proceedings of the 18th International Conference on the Foundations of Digital Games (FDG)*, 2023.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] L. de Moura and S. Ullrich, "The lean 4 theorem prover and programming language," in *International Conference on Automated Deduction (CADE)*, 2021.

[4] T. Yu, R. Zhang, K. Yang, M. Yasunaga *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," *arXiv preprint arXiv:1809.08887*, 2018.

[5] S. Sudhakaran, M. González-Duque, M. Freiberger, C. Glanois, E. Najarro, and S. Risi, "Mariogpt: Open-ended text2level generation through large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen, and J. Togelius, "Procedural content generation via machine learning (pcgml)," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018.

[7] J. Culberson, "Sokoban is pspace-complete," *Technical Report TR97-02, University of Alberta*, 1997.

[8] A. Junghanns and J. Schaeffer, "Sokoban: Enhancing general single-agent search methods using domain knowledge," *Artificial Intelligence*, vol. 129, no. 1–2, pp. 219–251, 2001.

[9] A. Guez, M. Mirza, K. Gregor, R. Kabra, S. Racanière, T. Weber, D. Raposo, A. Santoro, L. Orseau, T. Eccles, G. Wayne, D. Silver, T. Lillicrap, and V. Valdes, "An investigation of model-free planning: Boxoban levels," https://github.com/deepmind/boxoban-levels/, 2018.

[10] S. Racanière, T. Weber, D. P. Reichert, L. Buesing, A. Guez, D. Rezende, A. P. Badia, O. Vinyals, N. Heess, Y. Li, R. Pascanu, P. Battaglia, D. Hassabis, D. Silver, and D. Wierstra, "Imagination-augmented agents for deep reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[11] L. Orseau, L. H. S. Lelis, T. Lattimore, and T. Weber, "Single-agent policy tree search with guarantees," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[12] A. Garriga-Alonso, M. Taufeeque, and A. Gleave, "Planning behavior in a recurrent neural network that plays sokoban," in *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. [Online]. Available: https://openreview.net/forum?id=T9sB3S2hok

[13] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved variational inference with inverse autoregressive flow," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[14] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *International Conference on Learning Representations (ICLR)*, 2017.

[15] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[16] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.